

Exercice Docker-File Angular

Etape 1 : Cloner le repos du projet Angular

https://gitlab.com/jmlhmd/datacamp_docker_angular.git

Etape 2 : compiler le projet manuellement

Pour compiler le projet sur une image Docker, il faut créer un Dockerfile à double stages et procéder ainsi :

Stage 1 : Builder le projet

```
### STAGE 1: Build ###
```

- 1- Basez votre stage du Dockerfile sur `node:12.7-alpine`
- 2- Nommez cette image comme build
FROM `node:12.7-alpine AS build`
- 3- Utiliser le dossier `/usr/src/app` comme étant un répertoire de travail
WORKDIR `/usr/src/app`
- 4- Copiez les deux fichiers `package.json` `package-lock.json` au répertoire de travail
COPY `package.json package-lock.json .`
- 5- Lancer `npm install` à partir du répertoire de travail
RUN `npm install`
- 6- Copiez le contenu du projet
COPY `.`
- 7- Lancer `npm run build` sur la totalité du projet
RUN `npm run build`
- 8-

Stage 2 : lancer le projet moyennant nginx

```
### STAGE 2: Run ###
```

- 1- Basez votre stage du Dockerfile sur `nginx:1.17.1-alpine`
FROM `nginx:1.17.1-alpine`
- 2- Copiez le fichier de configuration `nginx.conf` vers le chemin approprié `/etc/nginx/nginx.conf`
COPY `nginx.conf /etc/nginx/nginx.conf`
- 3- Copier à partir du premier conteneur « build » l'application située dans le dossier `/usr/src/app/dist/aston-villa-app` vers le dossier de déploiement de nginx `/usr/share/nginx/html`
COPY `--from=build /usr/src/app/dist/aston-villa-app /usr/share/nginx/html`

Exercice Intégration continue d'un projet Angular

Création d'un Job Pipeline Jenkins

Ce pipeline doit :

Etape 1 :

récupérer le contenu du repository git

```
stage ('Clone Stage') {  
    steps {  
        git https://gitlab.com/jmlhmd/datacamp\_docker\_angular.git '  
    }  
}
```

Etape 2 :

ajouter une fonction qui appelle un script shell dans le pipeline: cette fonction a pour role de récupérer le numéro de version du build.

```
def getVersion(){  
    def version = sh returnStdout: true, script: 'git rev-parse --short HEAD'  
    return version  
}
```

puis appeler la fonction getVersion dans le bloc environment pour accorder la valeur retournée par cette fonction à une variable d'environnement

```
environment {  
    DOCKER_TAG = getVersion()  
}
```

Etape 3 :

builder l'image Docker de l'application

Le nom de l'image doit avoir la structure suivante :

Nom_utilisateur_github/tag:version

Pour ne pas avoir de problèmes liés au push de l'image sur github

```
stage ('Docker Build') {  
    steps {  
        sh 'docker build -t jmlhmd/image_name:${DOCKER_TAG}.'  
    }  
}
```

Etape 4 :

Ajouter un stage pour le déploiement (Push) de l'image docker sur dockerhub.

```
stage ('DockerHub Push') {  
    steps {  
        sh 'sudo docker login -u username -p password'  
        sh 'sudo docker push jmlhmd/image_name:${DOCKER_TAG}'  
    }  
}
```

Pour ne pas mettre les mots de passe en claire Il faut procéder ainsi

Jenkins

Jenkins

docker-ansible-demo

Pipeline Syntax

Back

Snippet Generator

Declarative Directive Generator

Declarative Online Documentation

Steps Reference

Global Variables Reference

Online Documentation

Examples Reference

IntelliJ IDEA GDSDL

Overview

This Snippet Generator helps you generate Jenkins Pipeline snippets. You can configure it, copy the snippet, and paste it into your Jenkinsfile. You can also paste the whole snippet or just the steps, leaving them at default values.

Steps

Sample Steps

libraryResource: Load a resource file from a shared library
load: Evaluate a Groovy source file into the Pipeline script
lock: Lock shared resource
mail: Mail
milestone: The milestone step forces all builds to go through in order
node: Allocate node
parallel: Execute in parallel
powershell: PowerShell Script
properties: Set job properties
pwd: Determine current directory
pwsd: PowerShell Core Script
readFile: Read file from workspace
readTrusted: Read trusted file from SCM
resolveScm: Resolves an SCM from an SCM Source and a list of candidate target branch names
retry: Retry the body up to N times
script: Run arbitrary Pipeline script
sh: Shell Script
sleep: Sleep
stage: Stage
stash: Stash some files to be used later in the build
step: General Build Step
svn: Subversion
timeout: Enforce time limit
timestamps: Timestamps
tm: Expand a string containing macros
tool: Use a tool from a predefined Tool Installation
unstable: Set stage result to unstable
unstash: Restore files previously stashed
validateDeclarativePipeline: Validate a file containing a Declarative Pipeline
waitForQualityGate: Wait for SonarQube analysis to be completed and return quality gate status
waitUntil: Wait for condition
warnError: Catch error and set build and stage result to unstable
withAnt: With Ant
withCredentials: Bind credentials to variables
withEnv: Set environment variables
withGradle: WithGradle
withSonarQubeEnv: Prepare SonarQube Scanner environment
wrap: General Build Wrapper
writeFile: Write file to workspace
ws: Allocate workspace

log out

Jenkins

Jenkins

docker-ansible-demo

Pipeline Syntax

Back

Snippet Generator

Declarative Directive Generator

Declarative Online Documentation

Steps Reference

Global Variables Reference

Online Documentation

Examples Reference

IntelliJ IDEA GDSDL

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain: Global credentials (unrestricted)

Kind: Secret text

Scope: Global (Jenkins, nodes, items, all child items, etc)

Secret:

ID:

Description:

Add Cancel

Add

Jenkins > docker-ansible-demo > Pipeline Syntax

Declarative Online Documentation
Steps Reference
Global Variables Reference
Online Documentation
Examples Reference
IntelliJ IDEA GDSDL

them at default values.)

Steps

Sample Step: withCredentials: Bind credentials to variables

Secret values are masked on a best-effort basis to prevent accidental disclosure. Multiline secrets, such as the contents of a SSH private key file, are not masked. See the inline help for details and usage guidelines.

Bindings

Secret text

Variable: dockerHubPwd

Credentials: docker-hub Add

Delete

Add

Generate Pipeline Script

```
withCredentials([string(credentialsId: 'docker-hub', variable: 'dockerHubPwd')]) {
    // some block
}
```

```
stage('DockerHub Push') {
    steps {
        withCredentials([string(credentialsId: 'mydockerhubpassword', variable:
'DockerHubPassword')]) {
            sh 'sudo docker login -u jmlhmd -p ${DockerHubPassword}'
        }
        sh 'sudo docker push jmlhmd/image_name:${DOCKER_TAG}'
    }
}
```

Etape 5 :

Maintenant, une fois que l'image est portée sur dockerhub, passons à utiliser ssh pour tous ce qui est opération de déploiement continue

Installer dans un premier temps les plugins SSH Credentials Plugin, SSH Agent Plugin et SSH Pipeline Steps

Créer par la suite un couple de clés ssh sur la machine de build et relier la clé publique aux paramètres d'accès à la machine de recette.

Une fois nous avons la clé private qui nous permet d'accéder à l'environnement de recette, nous utilisons son contenu et nous créons un credential (Vagrant_ssh) dans jenkins pour que ce dernier puisse accéder via ssh à l'environnement de recette.

```
stage('Deploy') {
    steps{
        sshagent(credentials: ['Vagrant_ssh']) {
            sh "ssh user@Ip_Recette"
            sh "ssh user@Ip_Recette 'sudo docker run \"image_name:${DOCKER_TAG}\"'"
        }
    }
}
```

PipeLine final:

```
pipeline {
  agent any
  tools{ jdk 'JDK8' }
  environment {
    JAVA_HOME = '/usr/lib/jvm/java-1.8.0-openjdk-amd64'
    DOCKER_TAG = getVersion()
  }
  stages {

    stage ('Clone Stage') {
      steps {
        git https://gitlab.com/jmlhmd/datacamp\_docker\_angular.git
      }
    }
    stage ('Docker Build') {
      steps {
        sh 'docker build -t jmlhmd/image_name:${DOCKER_TAG}.'
      }
    }
    stage ('DockerHub Push') {
      steps {
        withCredentials([string(credentialsId: 'mydockerhubpassword', variable:
'DockerHubPassword')]) {
          sh 'sudo docker login -u jmlhmd -p ${DockerHubPassword}'

          sh 'sudo docker push jmlhmd/image_name:${DOCKER_TAG}'
        }
      }
    }
    stage ('Deploy') {
      steps{
        sshagent(credentials: ['Vagrant_ssh']) {
          sh "ssh user@Ip_Recette"
          //sh "scp target/hello-world-app-1.0-SNAPSHOT.jar vagrant@192.168.1.201:/home/vagrant"
          sh "ssh user@Ip_Recette 'sudo docker run \"image_name:${DOCKER_TAG}\"'"
        }
      }
    }
  }
}

def getVersion(){
  def version = sh returnStdout: true, script: 'git rev-parse --short HEAD'
  return version
}
```

Connexion ssh à une VM Ubuntu

Création de l'environnement de Recette :

On va utiliser une VM ubuntu sur laquelle doit être installer le serveur SSH

- On démarre dessus le service ssh

service ssh status

service ssh start

Sur l'environnement de build ("JENKINS"), on crée une paire de clé lui permettant de se connecter à l'environnement de recette et spécifiquement au serveur ssh via la commande :

ssh-keygen -t rsa -b 2048

On reprend par la suite sur l'environnement de recette et crée un nouvel utilisateur. Il faut que le user créé dispose de privilèges et qu'il soit sur le groupe des root

sudo adduser admin

usermod -aG sudo admin

Si vous utilisez l'outil vagrant, vous pouvez vous contenter de l'utilisateur « vagrant »

On reprend par la suite sur l'environnement de build et on essaie d'envoyer la clé public à l'environnement de recette.

ssh-copy-id [admin@lp_address](#)

ou aussi

ssh-copy-id [vagrant@lp_address](#)

On saisit le mot de passe et on valide

Nous avons ainsi un utilisateur qui peut se connecter moyennant la clé rsa au serveur depuis l'environnement de build.